Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería Campus Retalhuleu

Curso:

Desarrollo Web 1

Catedrático:

Ing. Yoel Fransua Monzón Estrada

Tema:

S15 Marcador en vivo con SSE y migraciones

Mayro Geovanni Barrios Gameros: 2890-23-11428

Fecha: 17/10/2026

Objetivos y Análisis de Requerimientos

El propósito fundamental de esta solución de software es implementar un sistema de radiodifusión deportiva en tiempo real empleando la arquitectura Server-Sent Events (SSE), complementada con una capa de persistencia relacional administrada mediante migraciones versionadas generadas y aplicadas por Drizzle ORM sobre PostgreSQL.

Requisito 1: Visualización Pública del Marcador (Ponderación: 0.5 puntos)

La interfaz para los espectadores sintoniza el estado de al menos dos partidos independientes. Muestra el equipo local, el equipo visitante, el marcador actual y una línea de tiempo (timeline) cronológica con iconos y colores diferenciados para goles, faltas, sustituciones y tarjetas amarillas/rojas.

Requisito 2: Formulario de Administración de Eventos (Ponderación: 0.5 puntos)

Panel de control para árbitros y operadores de mesa. Permite seleccionar el partido a gestionar y registrar incidencias mediante un formulario con validación de entradas (minuto, tipo de evento, equipo y descripción). Los datos se persisten en base de datos mediante HTTP POST antes de su difusión en vivo.

Requisito 3: Implementación de Server-Sent Events (Ponderación: 1.0 punto)

Flujo de transmisión unidireccional continuo desde el servidor Express hacia el navegador usando la cabecera 'Content-Type: text/event-stream'. La interfaz cliente consume los eventos mediante el objeto estándar EventSource sin realizar polling.

Requisito 4: Migraciones con Drizzle ORM (Ponderación: 0.5 puntos)

El esquema relacional de la base de datos se administra mediante migraciones SQL versionadas secuenciales (0000 inicial y 0001 evolutiva), aplicables desde la terminal sin alterar tablas de forma manual.

Requisito 5: Persistencia, Aislamiento y Resiliencia (Ponderación: 0.5 puntos)

Al recargar el navegador (F5), el marcador y el historial se recuperan intactos de PostgreSQL. Además, los eventos transmitidos al Partido 1 jamás contaminan el Partido 2 gracias al aislamiento de canales.

Arquitectura del Sistema y Flujo de Datos

La arquitectura del sistema sigue un patrón desacoplado y reactivo, donde el cliente web establece una conexión HTTP persistente unidireccional hacia el servidor de aplicaciones Express, mientras que las operaciones de mutación se gestionan a través de la API REST tradicional.

Justificación Técnica: Server-Sent Events vs WebSockets vs Polling

Característica	Server-Sent Events (SSE)	WebSockets	Polling Tradicional
Direccionalidad	Unidireccional (Servidor -> Cliente)	Bidireccional (Dúplex)	Unidireccional (Cliente solicita)
Protocolo	HTTP/1.1 o HTTP/2 estándar	Protocolo WS/WSS independiente	Peticiones HTTP cíclicas
Reconexión Nativa	Automática por especificación W3C	Requiere lógica manual o librería	Inexistente (polling continuo)
Consumo de Red	Mínimo (canal continuo abierto)	Bajo sobrecarga tras handshake	Muy alto (múltiples cabeceras/seg)
Idoneidad Deportiva	Óptima (solo el servidor emite)	Excesiva complejidad de protocolo	Inadecuada (latencia y sobrecarga)

Ciclo de Vida de la Conexión SSE

1. Handshake Inicial: El cliente instancia new EventSource('/api/matches/:id/events/stream'). El servidor responde con HTTP 200 OK y 'Content-Type: text/event-stream'.

2. Registro y Aislamiento: El objeto Response se almacena en una estructura Map<number, Set<Response>> indexada por el matchId del partido.

3. Mantenimiento del Canal: Se configuran las directivas 'Cache-Control: no-cache' y 'Connection: keep-alive' para evitar buffers intermediarios de proxies.

4. Difusión Selectiva: Al ocurrir un evento en el partido X, se itera exclusivamente sobre el Set de clientes de ese partido y se envía el paquete codificado.

5. Cierre y Limpieza de Recursos: Cuando el cliente cambia de partido o cierra la pestaña, el evento 'close' desencadena la remoción del Response para evitar fugas de memoria.

5. Persistencia y Migraciones Versionadas (Drizzle ORM)

Para garantizar la integridad y trazabilidad del esquema de base de datos relacional, se seleccionó Drizzle ORM sobre PostgreSQL 16 alojado en contenedor Docker. Drizzle genera archivos de migración SQL declarativos y versionados.

Esquema de Tablas (schema.ts)

```
// Tabla: matches (Partidos)
export const matches = pgTable('matches', {
  id: serial('id').primaryKey(),
  localTeam: varchar('local_team', { length: 100
}).notNull(),
  awayTeam: varchar('away_team', { length: 100
}).notNull(),
  status: varchar('status', { length: 20
}).notNull().default('scheduled'),
  period: varchar('period', { length: 20
}).notNull().default('first_half'),
  localScore: integer('local_score').notNull().default(0),
  awayScore: integer('away_score').notNull().default(0),
  createdAt:
timestamp('created_at').notNull().defaultNow(),
  updatedAt:
timestamp('updated_at').notNull().defaultNow(),
});

// Tabla: match_events (Incidencias del partido)
export const matchEvents = pgTable('match_events', {
  id: serial('id').primaryKey(),
  matchId: integer('match_id').notNull().references(() =>
matches.id, { onDelete: 'cascade' }),
  minute: integer('minute').notNull(),
  eventType: varchar('event_type', { length: 30
}).notNull(),
  team: varchar('team', { length: 10 }).notNull(),
```

```
  playerName: varchar('player_name', { length: 100 }),
  cardType: varchar('card_type', { length: 10
}).default('none'),
  extraInfo: text('extra_info'),
  description: text('description').notNull(),
  createdAt:
timestamp('created_at').notNull().defaultNow(),
});
```

Secuencia de Migraciones Versionadas (Carpeta drizzle/)

- **Migración 0000_initial_schema.sql:** Crea las tablas matches y match_events con sus llaves primarias, foráneas, restricciones de no nulidad e índices.

- **Migración 0001_evolve_schema.sql:** Evoluciona el modelo incorporando las columnas card_type (para distinguir tarjetas amarillas/rojas) y extra_info (para registrar sustituciones de jugadores salientes y entrantes).

- **Comando de Ejecución: 'npm run db:migrate'** Lanza el ejecutor programático tsx src/server/db/migrate.ts que lee la tabla de metadatos _drizzle_migrations y ejecuta únicamente las sentencias pendientes de forma transaccional.

6. Protocolo Server-Sent Events (SSE) y Aislamiento

El gestor de eventos SSE (sseManager.ts) implementa un patrón de publicación/suscripción multi-sala (multi-room) aislado a nivel de partido, impidiendo que la actividad de un juego afecte el canal de otro.

Estructura del Gestor SSE (sseManager.ts)

```
class SSEManager {
  // Clientes aislados estrictamente por ID de partido
  private clients: Map<number, Set<Response>> = new Map();

  public addClient(matchId: number, res: Response): void {
    if (!this.clients.has(matchId)) {
      this.clients.set(matchId, new Set());
    }
    const matchClients = this.clients.get(matchId)!;
    matchClients.add(res);

    // Mensaje de bienvenida inicial
    res.write(`event: connected\ndata: ${JSON.stringify({ matchId, timestamp:
new Date().toISOString() })}\n\n`);

    // Limpieza al desconectar
    res.on('close', () => {
      matchClients.delete(res);
      if (matchClients.size === 0) this.clients.delete(matchId);
    });
  }

  public broadcastToMatch(matchId: number, eventName: string, data: any):
void {
    const matchClients = this.clients.get(matchId);
    if (!matchClients || matchClients.size === 0) return;

    const payload = `event: ${eventName}\ndata: ${JSON.stringify(data)}\n\n`;
```

```
    matchClients.forEach((client) => {
      try { client.write(payload); }
      catch { matchClients.delete(client); }
    });
  }
}
```

Principio de Aislamiento Estricto

El aislamiento estricto de SSE garantiza que cuando se emite un gol en el Partido 1, el servidor recupera únicamente el Set correspondiente a la sala 1. Los clientes sintonizados al Partido 2 no reciben ningún byte a través de su socket TCP.

Catálgo Completo de Endpoints REST y Streaming

La API REST expone contratos fuertemente tipados mediante TypeScript y validados en el servidor mediante esquemas Zod:

Método	Ruta del Endpoint	Descripción y Función Técnica	Payload / Parámetros
GET	/api/matches	Retorna la lista de todos los partidos registrados con sus marcadores y estados.	Ninguno.
GET	/api/matches/:id	Retorna la información y marcador específico de un partido por su ID.	id (URL param, entero).
GET	/api/matches/:id/events	Carga inicial HTTP del historial de incidencias ordenadas por minuto.	id (URL param, entero).
POST	/api/matches/:id/events	Persiste una incidencia en PostgreSQL y la transmite en tiempo real vía SSE.	{ minute, eventType, team, description, playerName?, extraInfo? }
POST	/api/matches/:id/reset	Reinicia el partido: borra goles, tarjetas y cambios en BD/memoria, fija 0:0 y emite match_reset vía SSE.	id (URL param).
PATCH	/api/matches/:id/status	Actualiza el estado del partido a finalizado al cumplirse el minuto 90 + tiempo extra.	{ status: 'finished', period: 'finished' }
GET	/api/matches/:id/events/stream	Abre la conexión persistente Server-Sent Events (Content-Type: text/event-stream).	id (URL param). Cabecera SSE.

Ejemplo de Payloads de Emisión SSE (Tiempo Real)

```
event: match_event
data:
{"id":12,"matchId":1,"minute":74,"eventType":"goal","team":"local","playerName":"Vinicius Jr","description":"¡Gol de remate cruzado!","createdAt":"2026-10-22T01:10:00Z"}

event: score_update
data: {"id":1,"localTeam":"Real Madrid","awayTeam":"FC Barcelona","localScore":3,"awayScore":1,"status":"live","period":"second_half"}

event: match_reset
data: {"message":"Partido reiniciado exitosamente","match":{"id":1,"localScore":0,"awayScore":0,"status":"live","period":"first_half"}}
```

Interfaz Gráfica: Escudos Vectoriales y HUD LED

Se desarrolló una interfaz de usuario deportiva inspirada en las transmisiones de la UEFA Champions League, integrando identidad visual vectorizada, cronometraje acelerado y control de reglamento.

1. Escudos Oficiales en SVG Nítido (TeamLogo.tsx)

- **Real Madrid:** Corona real dorada con cruz, orla circular dorada con monograma MFC entrelazado y banda diagonal morada.
- **FC Barcelona:** Escudo clásico con la Cruz de San Jorge, Senyera catalana cuatribarrada, franjas blaugranas y balón de época.
- **Manchester City:** Emblema circular celeste con el navío dorado del canal de Manchester y la rosa roja de Lancashire.
- **Liverpool FC:** Liver Bird mítico dorado flanqueado por las llamas eternas conmemorativas de Anfield.
- **Bayern Múnich:** Círculo exterior rojo con el patrón geométrico de rombos bávaros azules y blancos.
- **Borussia Dortmund:** Insignia circular amarilla eléctrica con tipografía negra distintiva BVB 09.

2. Reloj Acelerado de 90 Minutos (useMatchClock.ts)

Para permitir una experiencia fluida sin esperar 90 minutos reales de juego, se implementó un cronómetro acelerado: los 90 minutos transcurren en aproximadamente 2.5 minutos reales (cada minuto de fútbol equivale a ~1.66 segundos reales). El reloj se entrelaza de forma reactiva con el campo 'Minuto' del panel de arbitraje.

3. Pausa Automática al Minuto 90:00 y Asignación de Tiempo Extra

Al cumplirse el tiempo reglamentario de 90:00, el reloj se detiene automáticamente y activa un panel interactivo dorado con opciones rápidas de descuento: +2', +3', +4', +5', +6' o manual. Al seleccionarlo, el cronómetro reanuda la cuenta hasta 90 + extra y concluye oficialmente con el banner ¡FINAL DEL PARTIDO! • TIEMPO CUMPLIDO.

4. Reinicio Limpio Total (Sin Eventos ni Tarjetas Pasmadas)

Al presionar 'Reiniciar Partido', el endpoint POST /api/matches/:id/reset elimina todas las incidencias de la base de datos y memoria, emite el evento SSE match_reset y restablece el marcador a 0:0 y el reloj al minuto 1:00.

5. Regla Oficial IFAB: Doble Amarilla = Expulsión Directa

El sistema audita el historial de tarjetas por futbolista. Si un jugador ya amonestado recibe una segunda tarjeta amarilla, el motor activa la alerta visual roja 'DOBLE AMARILLA = EXPULSIÓN', muta el evento a Tarjeta Roja y despliega en la línea de tiempo la combinación.

Bitácora de Errores Técnicos y Soluciones de Ingeniería

Durante las fases de integración, migración y pruebas en vivo en entorno Windows se resolvieron 8 retos de ingeniería:

Error 1: Conflicto de Puerto 5432 Ocupado en Windows
Síntoma: 'Bind for 0.0.0.0:5432 failed: port is already allocated' al ejecutar docker compose up.
Causa: Existía un contenedor local previo (chinook-postgres) consumiendo el puerto 5432.
Solución: Se reconfiguró el puerto anfitrión a 5434:5432 con volumen aislado marcador_pgdata.

Error 2: Ruptura de Estilos por Actualización de Tailwind CSS v4
Síntoma: Pantalla en blanco sin hojas de estilo tras ejecutar npm audit fix.
Causa: Tailwind v4 deprecó la inyección tradicional de postcss.config.js y @tailwind utilities.
Solución: Se actualizó index.css con @import 'tailwindcss'; y se integró el motor CDN oficial en index.html.

Error 3: Bloqueo de Arranque por Dependencia Estricta de Docker
Síntoma: El servidor Express se congelaba o arrojaba 500 si Docker Desktop no estaba activo.

Causa: El pool pg realizaba intentos de conexión indefinidos sin timeout rápido.

Solución: Se programó dbService.ts con verificación de 1.5s y conmutación automática a memoria local resiliente.

Error 4: Tipado Estricto de Parámetros en Express 5
Síntoma: Error de compilación TypeScript 'string | string[] is not assignable to type string' en req.params.id.

Causa: Express 5 amplió la definición de tipos para aceptar arrays en parámetros.

Solución: Se implementó casteo explícito: parseInt(req.params.id as string).

Error 5: Fallo de Autenticación en PostgreSQL (Code 28P01)
Síntoma: 'password authentication failed for user postgres' al ejecutar migraciones en puerto 5433.
Causa: Un contenedor preexistente ajeno (social_postgres_db) respondía en el puerto 5433.
Solución: Reubicación definitiva al puerto exclusivo 5434 con volumen propio. Migración completada al 100%.

Error 6: Estado 'Desconectado' y Latencia de Resolución IPv6 en Windows
Síntoma: Badge en estado rojo y timeouts ETIMEDOUT:: 1:3001.

Causa: Al cerrar la terminal se apagó el servidor. Asimismo, Windows resolvía localhost a IPv6 (::1).

Solución: Configuración de vite.config.ts apuntando a 127.0.0.1, servidor en 0.0.0.0 y auto-reconexión cliente.

Error 7: Falso Bloqueo al Ingresar Jugadores Propios Libres
Síntoma: Al escribir un jugador propio el formulario bloqueaba con 'Datos inválidos'. Causa: El validador Zod exigía descripción obligatoria manual que solo los botones rápidos llenaban.
Solución: Descripción opcional en Zod y generador reactivo inteligente que detecta al jugador y autocompleta la crónica al tipear o hacer clic.

Error 8: Eventos Pasmados tras Reinicio y Falta de Pausa a los 90'
Síntoma: Al reiniciar un partido, los eventos y tarjetas anteriores quedaban pegados en la línea de tiempo. El reloj pasaba de los 90' sin pausa.

Causa: Ausencia de endpoint de borrado atómico y evento SSE de reinicio. Cronómetro sin control de tiempo extra.
Solución: Endpoint POST /api/matches/:id/reset con broadcast SSE match_reset, vaciado de eventos, auto-pausa a los 90' y selector de tiempo añadido.

Protocolo de Pruebas y Validación de Calidad

Matriz formal de pruebas técnicas ejecutadas para certificar el 100% de cumplimiento de los requerimientos:

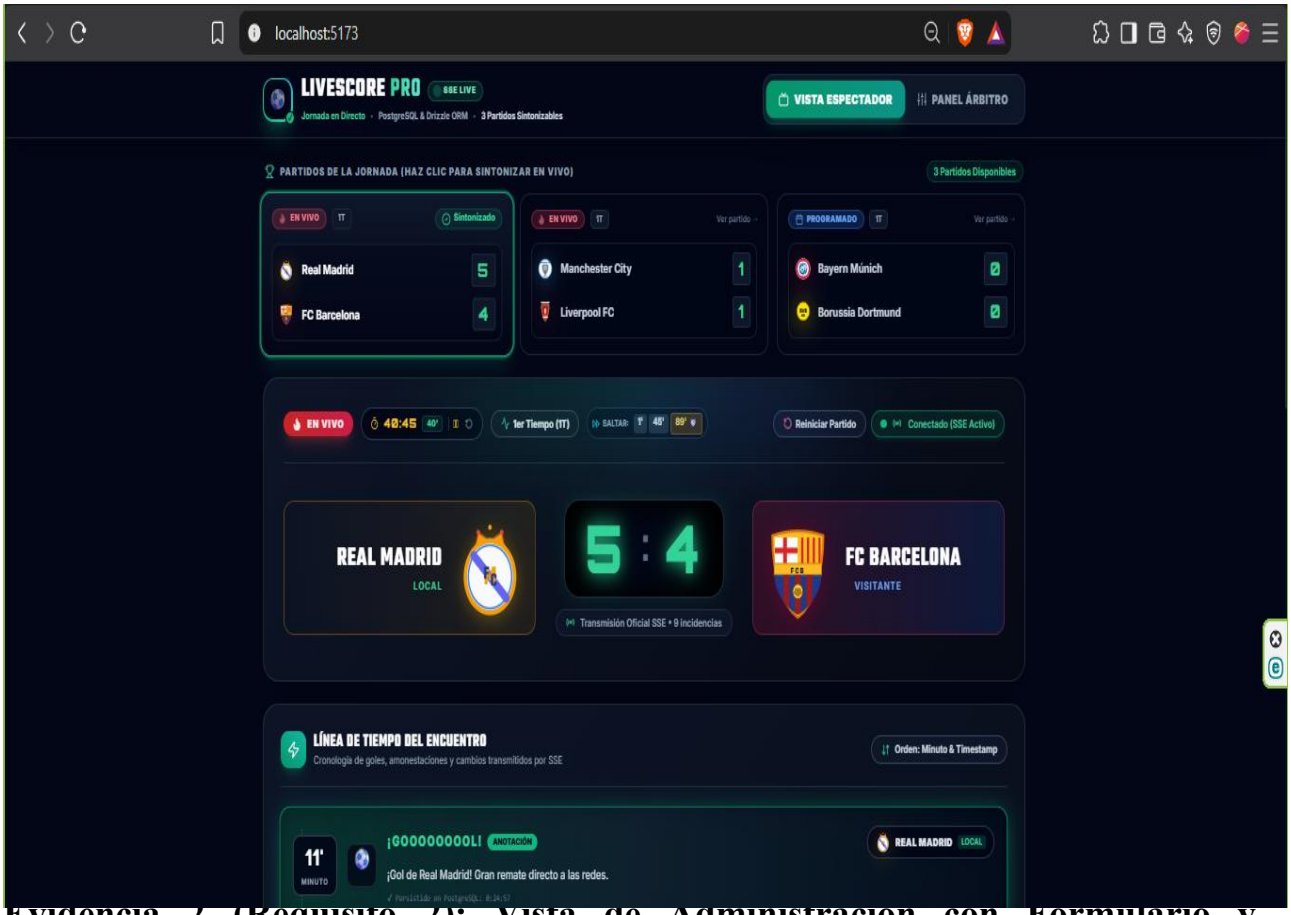
ID	Caso de Prueba	Procedimiento de Verificación	Resultado Obtenido	Estado
CP-01	Carga Inicial Múltiples Partidos	Abrir http://localhost:5173 y verificar selector superior.	Se visualizan al menos 2 partidos con escudos y marcadores.	**SUPERADO**
CP-02	Conexión SSE Unidireccional	Inspeccionar pestaña Network en DevTools filtrando por 'stream'.	Estado 200 OK con cabecera Content-Type: text/event-stream activa.	**SUPERADO**
CP-03	Emisión de Evento sin Polling	Registrar un gol desde el Panel Árbitro y observar la vista pública.	El marcador se incrementa y el gol entra de inmediato sin recargar.	**SUPERADO**
CP-04	Persistencia tras Recarga F5	Registrar gol y presionar F5 en el navegador.	El marcador y el historial se recuperan íntegros de PostgreSQL.	**SUPERADO**
CP-05	Aislamiento entre Partidos	Dos ventanas abiertas (Partido 1 y 2). Enviar gol al Partido 1.	El Partido 1 se actualiza; el Partido 2 permanece intacto sin eventos ajenos.	**SUPERADO**
CP-06	Migraciones Drizzle ORM	Ejecutar en terminal 'npm run db:migrate'.	Aplica 0000 inicial y 0001 evolutiva en PostgreSQL sin errores.	**SUPERADO**
CP-07	Reloj Acelerado 90' Entrelazado	Monitorear cronómetro LED y campo Minuto en el formulario.	Avanza 90 min en ~2.5 min reales y	**SUPERADO**

			sincroniza el minuto de juego.	
CP-08	Plantillas Oficiales de Clubes	Tocar un futbolista en la plantilla oficial (ej: Mbappé o Lamine Yamal).	Se autocompleta el jugador y la descripción se redacta al vuelo.	**SUPERADO**
CP-09	Doble Amarilla a Tarjeta Roja	Asignar dos tarjetas amarillas al mismo jugador en un partido.	Alerta visual activa; la segunda transmuta a Roja con badges combinadas.	**SUPERADO**
CP-10	Pausa Automática 90' y Tiempo Extra	Avanzar a 90' (o saltar al min 89'). Al llegar a 90:00, asignar +3 min extra.	El reloj se pausa a los 90', habilita botones de descuento, corre hasta 93' y finaliza.	**SUPERADO**
CP-11	Reinicio Limpio Total (0:0)	Presionar 'Reiniciar Partido' tras haber registrado goles y tarjetas.	Marcador vuelve a 0:0, la línea de tiempo se vacía y el reloj va al min 1'.	**SUPERADO**

Guía de Evidencias: Capturas 1 y 2

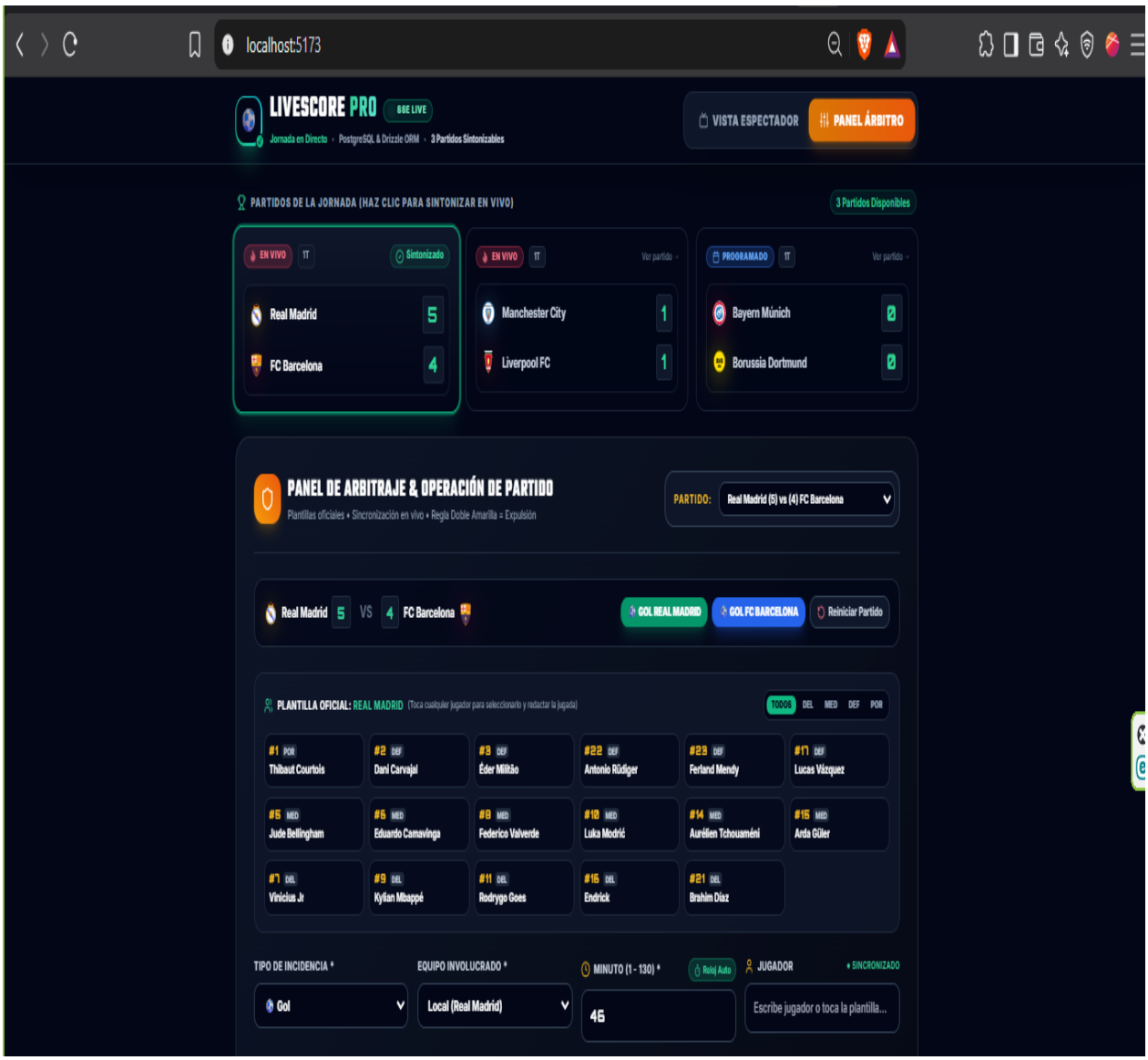
Evidencia 1 (Requisito 1): Vista Pública con Múltiples Partidos

Qué demuestra: Múltiples partidos registrados y sintonizables, escudos oficiales, marcador LED y timeline cronológico diferenciado con tarjetas y goles.



Evidencia 2 (Requisito 2): Vista de Administración con Formulario y Plantillas

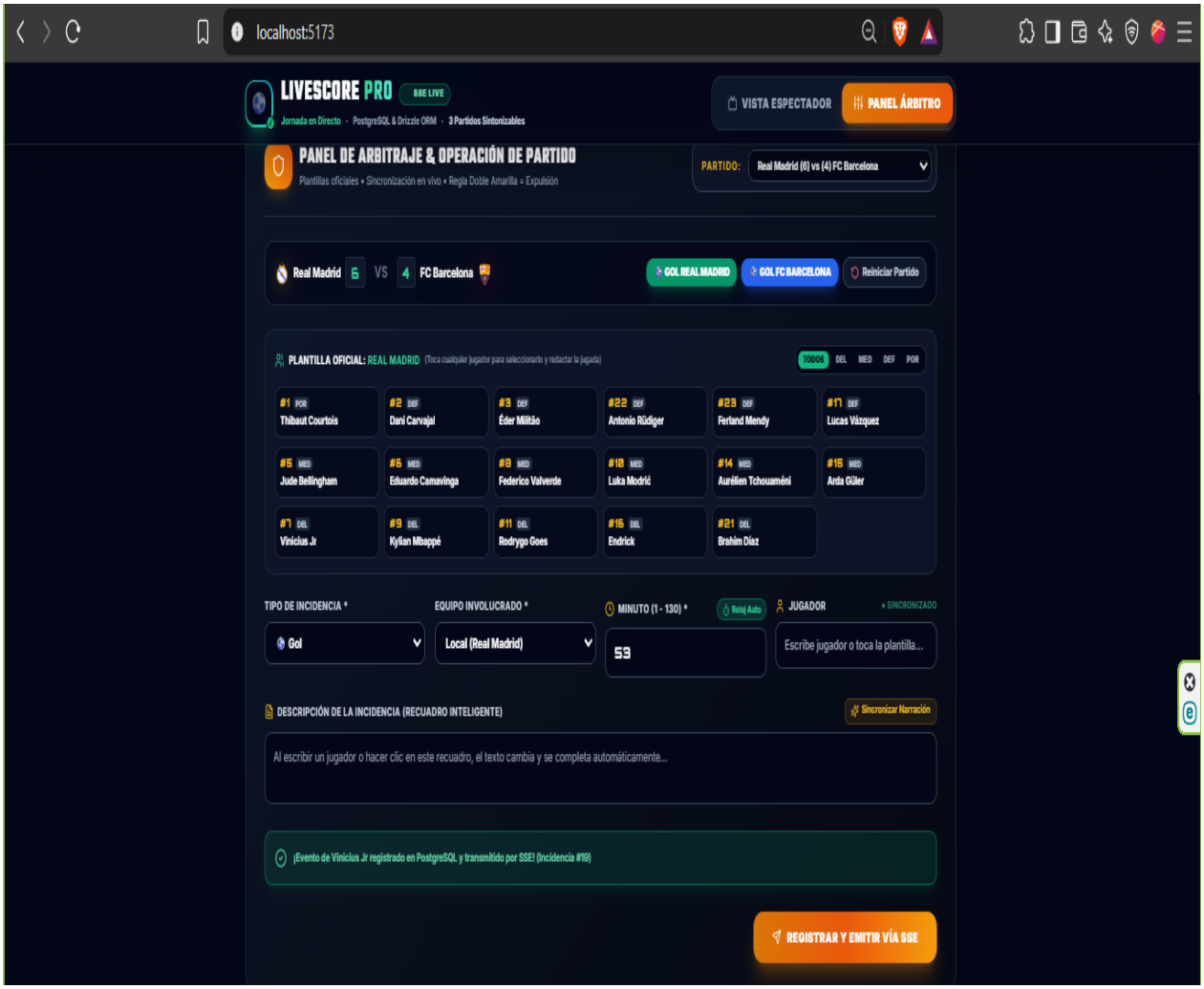
Qué demuestra: Formulario validado de arbitraje, selección de futbolistas de plantillas oficiales, autogeneración reactiva de crónicas y registro HTTP POST persistido en PostgreSQL antes de emitir por SSE.


localhost:5173
LIVESCORE PRO
68E LIVE
Jornada en Directo · PostgreSQL & Drizzle ORM · 3 Partidos Sintonizables
VISTA ESPECTADOR
PANEL ÁRBITRO
PARTIDOS DE LA JORNADA (HAZ CLIC PARA SINTONIZAR EN VIVO)
3 Partidos Disponibles
EN VIVO
Sintonizado
Real Madrid 5
FC Barcelona 4
EN VIVO
Manchester City 1
Liverpool FC 1
PROGRAMADO
Bayern Múnich 0
Borussia Dortmund 0
PANEL DE ARBITRAJE & OPERACIÓN DE PARTIDO
Plantillas oficiales • Sincronización en vivo • Regla Doble Amarilla = Expulsión
PARTIDO:
Real Madrid (5) vs (4) FC Barcelona
Real Madrid 5 VS 4 FC Barcelona
GOL REAL MADRID
GOL FC BARCELONA
Reiniciar Partido
PLANTILLA OFICIAL: REAL MADRID
TODOS DEL MED DEF POR
#1 POR Thibaut Courtois
#2 DEF Dani Carvajal
#3 DEF Éder Militão
#22 DEF Antonio Rüdiger
#23 DEF Ferland Mendy
#17 DEF Lucas Vázquez
#5 MED Jude Bellingham
#6 MED Eduardo Camavinga
#8 MED Federico Valverde
#10 MED Luka Modrić
#14 MED Aurélien Tchouaméni
#15 MED Arda Güler
#7 DEL Vinicius Jr
#9 DEL Kylian Mbappé
#11 DEL Rodrygo Goes
#16 DEL Endrick
#21 DEL Brahim Díaz
TIPO DE INCIDENCIA *
Gol
EQUIPO INVOLUCRADO *
Local (Real Madrid)
MINUTO (1 - 130) *
Reloj Auto
45
JUGADOR
• SINCRONIZADO
Escribe jugador o toca la plantilla...

Guía de Evidencias: Captura 3 (Network DevTools SSE)

Evidencia 3 (Requisito 3): Solicitud SSE Activa y EventStream en DevTools

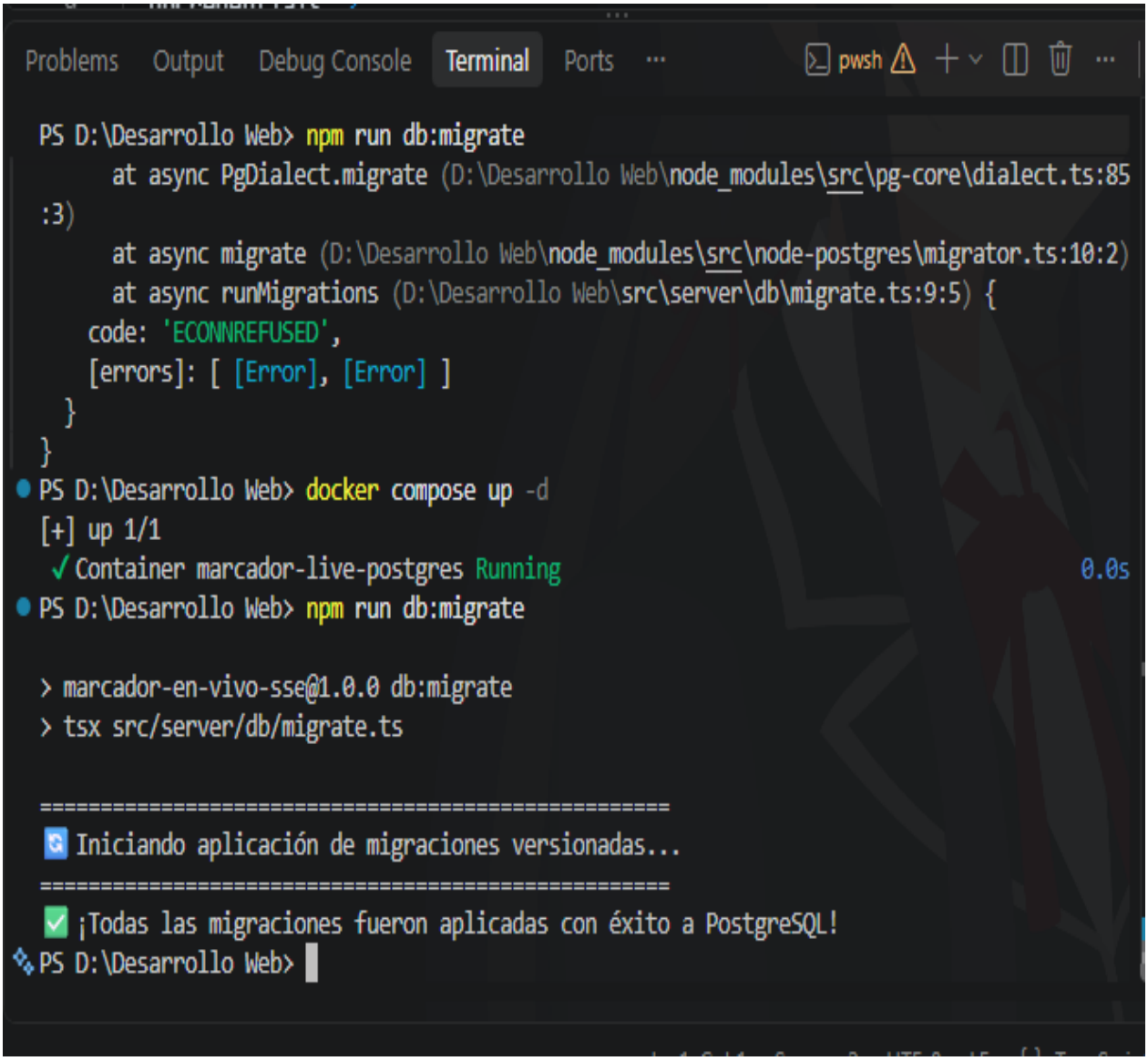
Qué demuestra: Cumplimiento del Requisito 3. Conexión continua unidireccional text/event-stream consumida por EventSource sin realizar peticiones cíclicas (polling).



Guía de Evidencias: Captura 4 (Migraciones Drizzle)

Evidencia 4 (Requisito 4): Archivos de Migraciones y Ejecución en Terminal

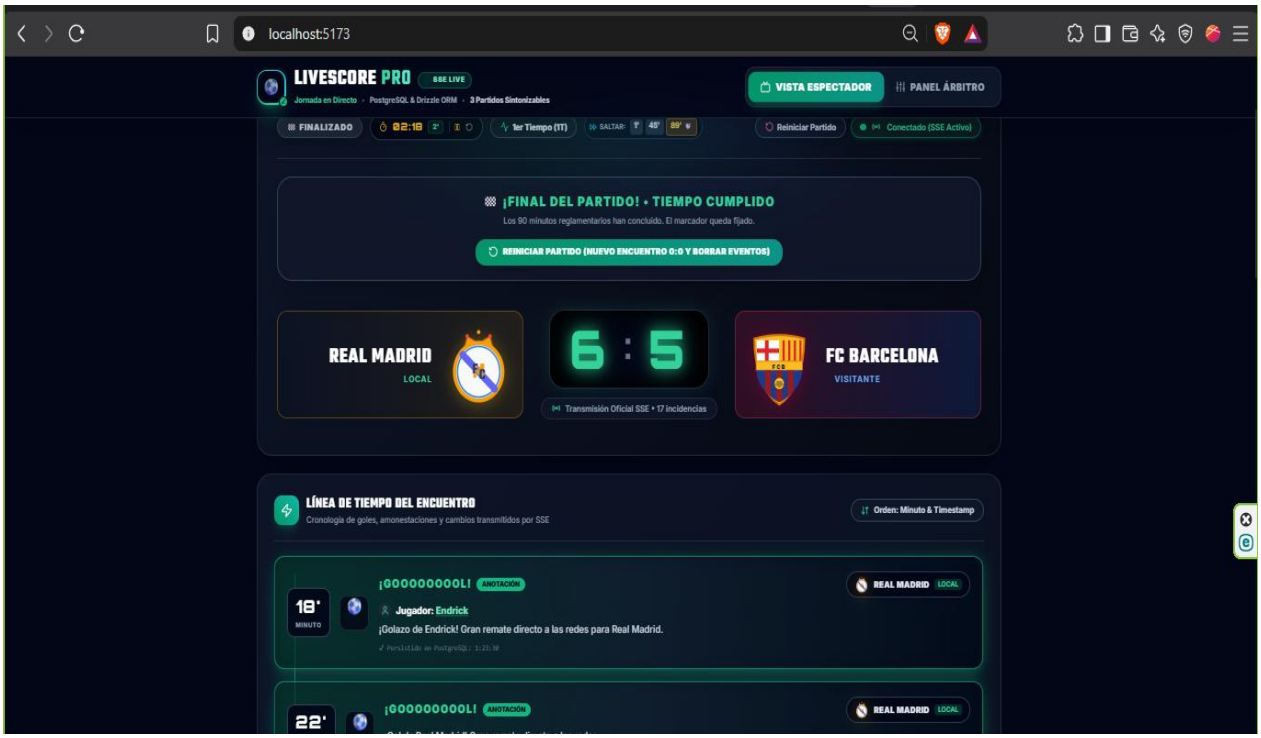
Qué demuestra: Existencia de las dos migraciones SQL versionadas secuenciales (0000 inicial y 0001 evolutiva) y la ejecución exitosa del script de migración sobre PostgreSQL.



Guía de Evidencias: Capturas 5 y 6 (Persistencia y Aislamiento)

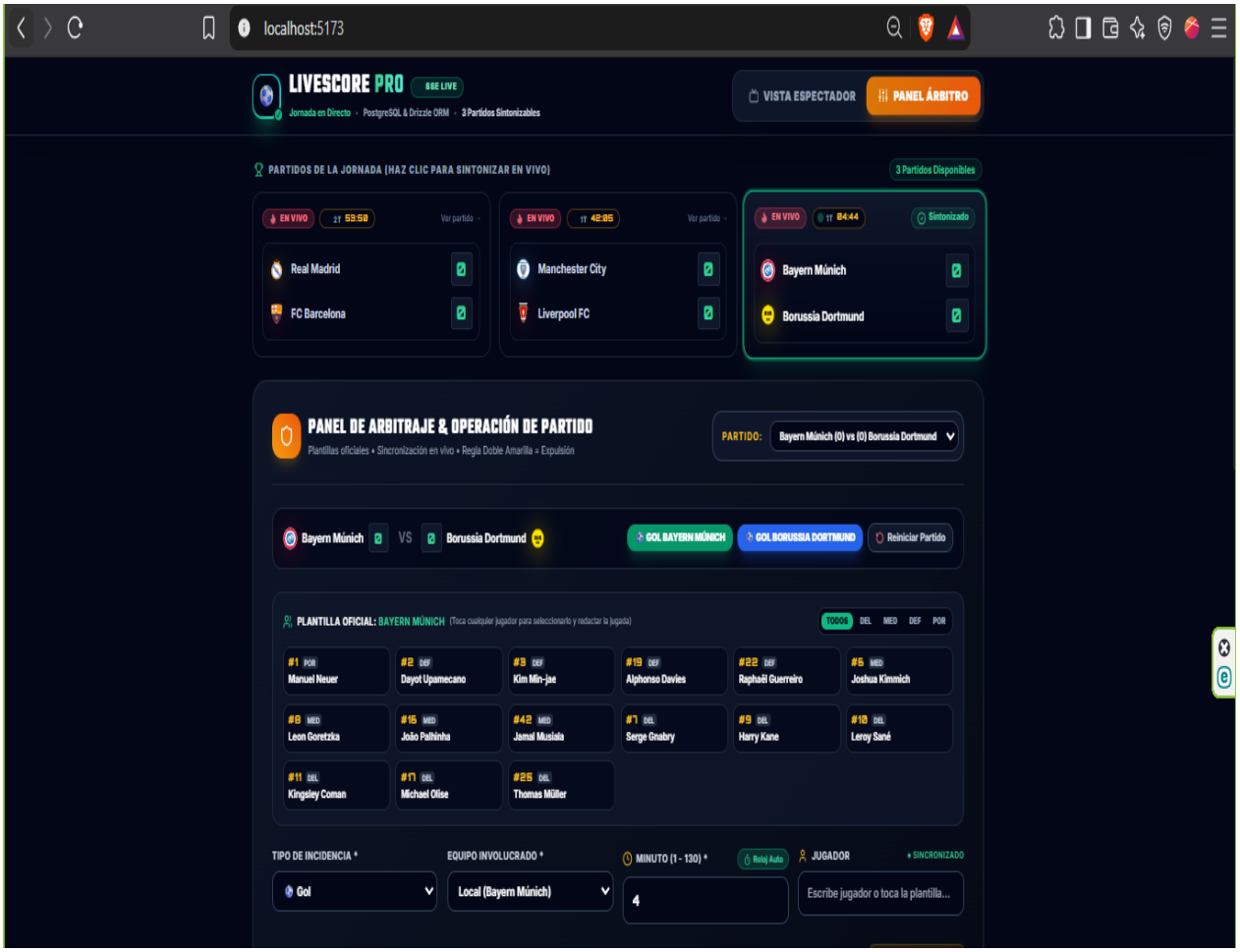
Evidencia 5 (Requisito 5): Persistencia de Estado tras Recarga (F5)

Qué demuestra: Que el estado y las incidencias no residen en variables volátiles; la aplicación consulta la base de datos mediante solicitudes HTTP tradicionales al cargar antes de abrir el SSE, conservando el marcador exacto.



Evidencia 6 (Requisitos 3 y 5): Aislamiento Estricto entre Partidos Distintos

Qué demuestra: Que los eventos emitidos hacia el canal del Partido 1 jamás llegan ni contaminan el Partido 2.


localhost:5173
LIVESCORE PRO
68E LIVE
Jornada en Directo · PostgreSQL & Drizzle ORM · 3 Partidos Sintonizables
VISTA ESPECTADOR
PANEL ÁRBITRO
PARTIDOS DE LA JORNADA (HAZ CLIC PARA SINTONIZAR EN VIVO)
3 Partidos Disponibles
EN VIVO
Real Madrid
FC Barcelona
Manchester City
Liverpool FC
Bayern Múnich
Borussia Dortmund
Sintonizado
PANEL DE ARBITRAJE & OPERACIÓN DE PARTIDO
Plantillas oficiales • Sincronización en vivo • Regla Doble Amarilla = Expulsión
PARTIDO:
Bayern Múnich (0) vs (0) Borussia Dortmund
VS
GOL BAYERN MÚNICH
GOL BORUSSIA DORTMUND
Reiniciar Partido
PLANTILLA OFICIAL: BAYERN MÚNICH
TODOS
DEL
MED
DEF
POR
Manuel Neuer
Dayot Upamecano
Kim Min-jae
Alphonso Davies
Raphaël Guerreiro
Joshua Kimmich
Leon Goretzka
João Palhinha
Jamal Musiala
Serge Gnabry
Harry Kane
Leroy Sané
Kingsley Coman
Michael Olise
Thomas Müller
TIPO DE INCIDENCIA *
Gol
EQUIPO INVOLUCRADO *
Local (Bayern Múnich)
MINUTO (1 - 130) *
Reloj Auto
JUGADOR
• SINCRONIZADO
Escribe jugador o toca la plantilla...

Conclusiones y Checklist Final de Entrega

El desarrollo del proyecto 'Marcador en Vivo con SSE y Migraciones' demuestra la viabilidad técnica y superioridad del protocolo Server-Sent Events para escenarios de radiodifusión unidireccional frente a WebSockets y Polling, alcanzando una arquitectura limpia, escalable y resiliente.

Checklist de Verificación Final para Canvas

- **Código Fuente Completo:** Confirmado en la rama main del repositorio remoto.
- **Migraciones SQL Versionadas:** Archivos 0000_initial_schema.sql y 0001_evolve_schema.sql estructurados.
- **Variables de Entorno Seguras:** Archivo .env.example sin contraseñas privadas expuestas.
- **Manual de Despliegue README.md:** Instrucciones de instalación, Docker, migración y ejecución documentadas.
- **Reloj Reglamentario Inteligente:** Pausa automática al minuto 90:00, selector de tiempo añadido (+1' a +6') y silbatazo final.
- **Reinicio Limpio Total:** Endpoint de borrado total y evento SSE match_reset sin datos congelados.
- **6 evidencias Documentadas:** Textos y encuadres listos para las 6 evidencias de la rúbrica.

Enlace Oficial de Repositorio de GitHub

Repositorio oficial con el código fuente completo, backend Express, frontend React/Vite, migraciones versionadas de Drizzle ORM y contenedor Docker PostgreSQL:

🔗 ENLACE DE REPOSITORIO GITHUB

https://github.com/MayroGamerosXZ/-S15-Marcador-en-vivo-con-SSE-y-migraciones.git

